THz-TDS Parameter Extraction Explained

https://gemini.google.com/app/fa6d20f11926d738

User prompt: This is an extraction code using different algorithms to find the ref index and thickness of two layers in a THz-TDS. Can you comment on every code block and tell me what it does, refer to every code block by a number.

Response: Working through parameter extraction code can be a bit dense, but this notebook is structured very logically. It is essentially building a physical simulation of a Terahertz (THz) pulse moving through a multi-layered material, and then using a gradient-based machine learning optimizer (PyTorch) to tweak the material's properties until the simulation matches your real-world experimental data.

Here is a breakdown of what every code block is doing, referenced by their execution order:

1. Notebook Styling

Sets the width of the Jupyter Notebook container to 90%. This simply utilizes more of your screen for wider code blocks and plots.

2. Matplotlib Backend

The `%matplotlib ipympl` magic command enables interactive plots within the notebook, allowing you to zoom and pan the generated graphs rather than just viewing static images.

3. Library Imports

Imports the standard tools needed for the task:

- `tqdm` for progress bars.
- `numpy` and `matplotlib.pyplot` for math arrays and plotting.
- Custom modules (`Simulate` and `data_quartzsi`) that bring in your raw experimental data: the time vector (`t_r`), the reference pulse (`A_r`), and the sample pulse (`A_s`).

4. Synthetic Reference Pulse

Defines a function (`simulate_reference`) to generate a purely synthetic, mathematically ideal THz reference pulse. It uses a Gaussian derivative profile. This is useful if you want to test the algorithm without using real experimental noise.

5. Physical Simulation (Transfer Matrix)

This is the core physics engine of the script. It defines three functions to simulate how the THz wave propagates through the material:

- `rts`: Calculates the Fresnel reflection and transmission coefficients for a single layer boundary.
- `RTm`: Iteratively applies the `rts` function across m layers to calculate the total transfer matrix of the entire sample.
- `simulate_parallel`: Takes an input pulse, converts it to the frequency domain via Fast Fourier Transform (FFT), applies the transfer matrix, adds artificial noise, and converts it back to the time domain (IFFT).

6. Data Mode Toggle

Sets a boolean flag (`simulate_data = False`). This acts as a switch to tell the notebook whether to use the synthetic data generated in Block 4, or real experimental data.

7. Data Loading Setup

A conditional block that acts on the toggle from Block 6. Because it is set to `False`, the script bypasses the synthetic generation and loads your imported experimental data (`A_r` and `A_s`) into the variables `x_exp` and `true_x_exp`.

8. MAE Loss Function

Defines a simple Mean Absolute Error (MAE) function. This is used to calculate the raw, linear difference between the true experimental pulse and the simulated pulse.

9. Baseline Simulation & Plotting

Sets up an initial "best guess" for the refractive index (n), extinction coefficient (k), and thickness (d) of the two layers. It runs one forward simulation using these guesses, calculates the initial error, and plots it. This gives you a visual baseline of how far off your initial guesses are from the real experimental pulse.

10. Log-Cosh Loss Function

Defines a Log-Cosh loss function using PyTorch. Log-Cosh mathematically approximates the Mean Absolute Error but is smooth and differentiable everywhere. This makes it much easier for the gradient-based optimizer to calculate exactly how to adjust the parameters without getting stuck.

11. The Optimization Loop

This is the heavy lifter. The `find_optimal_params` function:

- Initializes PyTorch tensors for your n, k, and d variables and sets `requires_grad=True` so the optimizer can track them.
- Sets up an Adam optimizer.

- Runs a loop for a set number of iterations. In each loop, it simulates the pulse, calculates the error using the Log-Cosh function, and backpropagates the error to tweak the n, k, and d values slightly closer to reality.
- Finally, it plots the loss trajectory over time to show how the model converged.

12. Parameter Extraction

A helper function (`nkds_from_trajectory`) that simply extracts and prints the final, optimized n, k, and d values from the trajectory history generated in Block 11.

13. Final Plotting Function

A helper function (`plot_pulses`) that takes your newly optimized parameters, runs one final simulation, and plots the resulting pulse on top of the real experimental pulse so you can visually verify the fit.

14. Iteration Count

Sets the variable for the number of optimization loops to 50,000.

15. Execution 1: First Guess

Executes the optimizer function from Block 11 using your first set of initial guesses and saves the history to `trajectory1`.

16. Execution 2: Second Guess

Executes the optimizer again, but with a slightly altered initial guess for the first layer (changing n to 1.90 and d to 900um). This is a common practice to ensure the algorithm finds the true global minimum and didn't just get stuck in a "local minimum" based on where it started.

17. Execution 3: Third Guess

Executes the optimizer a third time with yet another varied starting point ($n = 2.01$).

18. Execution 4 (Commented Out)

Code that was commented out, likely representing an abandoned attempt at a fourth starting guess.

19. Trajectory Aggregation

Gathers the successful optimization runs (`trajectory1`, `2`, and `3`) into a single list for easier programmatic comparison.

20. Output Cell

This block shows the printed output and final graph generated by evaluating the `trajectory1` results. It prints the optimized n, k, and d values alongside the final loss, and displays the loss convergence graph and the overlapping pulse plot.
